

ENDOGENOUS SIGNALS FAIL TO ALLOCATE: PER-TOKEN COMPUTE VALUE IS REAL, TRANSIENT, AND NOT LEARNABLE

Anonymous authors

Paper under double-blind review

ABSTRACT

Token-level adaptive compute usually keys off endogenous signals—attention entropy, router scores, confidence—assumed to track per-token difficulty. On arithmetic with graded difficulty we show this assumption fails three ways: an entropy-driven gate collapses to a constant, a tag-initialized gate inverts, and an unsupervised learned head is vacuous. Difficulty-supervised targets under a budget constraint do produce ordered gates, but a pre-registered shuffled-targets control shows the ordering carries no accuracy benefit: gate-grad targets trail permuted targets by 0.0008 held-out loss at 1.4B, and by at most 0.3 accuracy points at 11M. Measuring the field directly—the per-token gradient of loss with respect to each token’s compute gate—explains why. The field has a life cycle: real structure mid-training (permutation-controlled oracle), consumed by training, flat at convergence—and a causal test shows its structure is not trainable: chasing it online loses to random allocation of the same budget at every scale tested. The surviving recipe needs no allocator: token-level FFN dropout, where each token receives full FFN compute only inside randomly rotating windows. At 1.4B web-text fine-tuning it trails dense by 7.8% held-out loss (2 seeds) at 50% billed FFN training FLOPs, inside the pre-registered 2–10% band; a gather-based realization converts half the billed cost into a measured $1.35\times$ training throughput. Failures on code and template-dominated math are reported, not hidden.

Transformers spend most of their training compute on feed-forward blocks applied uniformly to every token at every step. We ask a simple question: does every token need the FFN at every step? The adaptive-compute literature answers with learned allocators—routers, entropy gates, difficulty heads—trained end-to-end to switch compute on where it “matters.” We first show this program fails on its own terms in a controlled testbed with per-token ground truth: endogenous signals collapse, invert, or stay vacuous (§2.2), supervised targets produce correctly ordered gates whose ordering carries no accuracy benefit, and a pre-registered shuffled-targets control shows target content is irrelevant at 11M and 150M scale. The positive half of the paper is a measurement that explains these failures and a recipe it motivates. We measure the per-token *value of compute*—the gradient of the loss with respect to each token’s FFN gate—and find it has a life cycle: permutation-controlled measurement shows real structure mid-training, consumed by training and flat at convergence (§2.4). A permutation-controlled oracle bounds the converged field’s realizable headroom, and a causal single-stage test shows that *chasing* the field online is worse than random allocation of the same budget—at 11M and, replicated, in the starved-150M regime where the field’s mid-life structure is demonstrably present (§4). What works is the degenerate allocator everyone skips: train with *token-level FFN dropout*—each token receives full FFN compute only inside randomly rotating windows—and deploy dense. At matched tokens this trails dense held-out loss by 5.3% at half the billed FFN training FLOPs on 11M and 150M models, and by 7.8% on 1.4B web-text fine-tuning—a claim we scope to compute-limited regimes per its pre-registered decision rule—requires no labels or routers, and composes with any attention architecture because it never touches attention.

Contributions.

1. **A three-way failure diagnosis** of endogenous allocation signals (collapse, inversion, vacuity), each with a mechanistic account (§2.2); a standard Mixture-of-Depths router fails analogously.
2. **Compute-value supervision**: targets derived offline from the trained model itself—per-token loss, and $\partial\mathcal{L}/\partial g$ in one backward pass—match label-supervised allocation, with a shuffled-targets control (and a pre-registered decision rule) separating the specific measured targets from supervised variation alone (§2.4, §4.2); the value field’s per-bin signature is measured at 42M and 150M, where the pre-registered migration prediction fails and the converged field is flat (§4.4).
3. **Iso-FLOPs methodology**: all method comparisons at matched *billed training FLOPs* (gated billing, forward + backward), with within-segment interpolation and a budget sweep; the workshop version’s “similar total budget” caveat is retired (§2.5).
4. **Scale replication, honestly scoped**: the failure taxonomy, the shuffled-control content-irrelevance result, and the causal field-chasing test all replicate at 150M (probe grid, starved regime); the dropout recipe’s parity claim is scoped to compute-limited regimes by its pre-registered decision rule (−5.3% loss at 50% of billed FFN FLOPs at 150M; −7.8% at 1.4B web-text fine-tuning; the boundary on template-dominated data is reported, not hidden) (§4.4, §4.5).
5. **Realization**: a tiered-FFN serving benchmark showing tokens/sec scaling with the mean gate, converting billed-FLOPs savings into wall-clock (§4.7).

1 RELATED WORK

Adaptive compute. Adaptive Computation Time (Graves, 2016), early-exit and cascades, and Mixture-of-Depths (Raposo et al., 2024) route tokens by learned signals; MoE+router work attaches learned routers to expert selection. Raposo et al. (2024) train routers from scratch; our MoD baseline on the language-model track inherits the resulting handicap—a router trained during a short fine-tune, not a full pretraining run—which we acknowledge as a baseline limitation, not a strawman.

Conditional computation and mixture-of-experts.

Entropy and confidence as uncertainty proxies. Draft-model disagreement in speculative decoding is a deployed, per-token difficulty signal: acceptance probability measures “hard for a small model,” and our gate-gradient targets are the same quantity computed for layers instead of models.

Resource-aware agents.

2 METHOD

2.1 ARCHITECTURE AND PER-TOKEN GATING

Each layer computes

$$x_2 = \text{LN}(x_1 + \text{attn}(x_1)), \quad x_3 = \text{LN}(x_2 + g_t \cdot \text{FFN}(x_2)), \quad (1)$$

where $g_t \in [0, 1]$ is a per-token, per-layer gate produced by a small per-layer *DifficultyHead* (a two-layer MLP on the post-attention state x_2 , sigmoid-squashed). Compute is billed proportionally: a position gated at g is billed g times the per-token FFN cost; attention is always paid in full. All variants share Eq. (1) and the accounting, so comparisons isolate the gating signal. We gate the FFN only, never the attention’s keys or values: skipping a token’s FFN is safe because downstream tokens still attend to it, whereas skipping K/V irreversibly deletes information that later positions may need.

2.2 WHY ENDOGENOUS SIGNALS FAIL: THREE DIAGNOSTICS

End-to-end training offers the gate one channel of feedback: the task loss. *Collapse*: a signal with no difficulty information (attention entropy, $r \approx -0.06$ per token with difficulty) drives the gate to a constant. *Inversion*: a ground-truth initialization makes the gate vary, but easy problems are memorizable, so their cross-entropy gradient is largest and training allocates them *more* compute (per-problem gate-difficulty $r = -0.59$). *Vacuity*: a head trained only through the task loss receives no signal about what difficulty is; the gate stays flat and unstable across seeds. We report the two iteration-1 quantities as distinct: per-token entropy-difficulty correlation $r \approx -0.06$, and per-problem entropy-gate-difficulty $r = 0.23$ (the gate passes the tag embedding through a bottleneck; the two correlations measure different objects).

2.3 EXPLICIT DIFFICULTY SUPERVISION WITH A BUDGET CONSTRAINT

Training is two-stage. **Stage 1** trains the body conventionally with the gate fixed at 1 and checkpoints it. **Stage 2** attaches the DifficultyHeads and minimizes

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda_{\text{mse}} \frac{1}{N} \sum_t (\sigma(s_t) - \tau_t)^2 + \lambda_{\text{budget}} (\bar{g} - 0.5)^2, \quad (2)$$

with $\lambda_{\text{mse}} = 5.0$, $\lambda_{\text{budget}} = 0.5$; the body trains at a near-frozen learning rate ($10^{-2} \times$ the heads'). Stage 1 is shared by every fork of the method (all Stage-2 arms load the same checkpoint per seed), so method comparisons are paired by body.

2.4 MEASURING THE VALUE OF COMPUTE

Where difficulty labels are absent (all natural-language settings), targets τ_t are *measured offline* from the Stage-1 checkpoint:

$$\text{score}_t = - \sum_{\ell} \frac{\partial \mathcal{L}}{\partial g_{t,\ell}} \Big|_{g=1}, \quad (3)$$

the marginal loss reduction from one unit of FFN compute at token t , obtained for all tokens and layers in a single forward-backward pass with gates as differentiable leaves. Scores ≤ 0 are *irreducible* (compute does not help there) and receive the floor target 0.1; positive scores are quartile-binned to 0.2/0.4/0.6/0.8. This is the paper's key conceptual move: tags label *difficulty*; Eq. (3) measures the *value* of compute. The two are different objects, and the difference is testable: score-loss correlation is near zero in our runs—loss says “hard,” the gate gradient says “helpable”—and a token's tag bin does not constrain its score (a hard bin mixes irreducible tokens with worth-treating ones). We therefore expect, and report, that compute-value supervision does *not* order gates by difficulty bins at the testbed scale: bin-level spread is not its target. The workshop-era inversion—end-to-end training overfeeding easy tokens—is the same phenomenon seen from the other side: training chases reducibility wherever it lives, but uncontrollably; the budget term is what makes chasing value safe. A loss-quantile variant (the proxy-difficulty ablation) bins tokens by per-token loss; a *shuffled* control permutes targets within each sequence, testing whether any varying supervision would do.

First-order oracle allocation. The same measurements give a reference point: sorting per-(token, layer) scores and filling until the budget is exhausted yields the first-order-optimal allocation under a *frozen* body, evaluated zero-shot by overriding the gates at inference. We label it accordingly: it bounds what allocation alone buys without retraining. Trained methods may exceed it—their bodies adapt to their gates—and we interpret that as adaptation compounding allocation, not as a paradox.

2.5 ISO-FLOPS TRAINING PROTOCOL

Every run logs cumulative *billed training FLOPs*: the gated billing of §2.1 applied to forward, with backward billed at twice forward (total $3\times$). Methods are compared at matched

Table 1: P0: paired Stage-2 arms from the same Stage-1 body per seed (3 seeds, mean $\pm$ std). Billed training FLOPs are identical across the bottom two arms by construction (the oracle pass is billed to both); the tag arm has no oracle pass.

Stage-2 arm	Acc	Balanced	MFLOPs/tok	Gate spread
Tag targets (labels)	74.8 $\pm$ 1.3	83.6 $\pm$ 0.8	19.4 $\pm$ 0.1	0.21 $\pm$ 0.06
Gate-grad (measured value)	74.5 $\pm$ 1.0	83.4 $\pm$ 0.7	19.2 $\pm$ 0.0	0.08 $\pm$ 0.01
Shuffled control	74.7 $\pm$ 1.1	83.5 $\pm$ 0.7	19.3 $\pm$ 0.0	0.06 $\pm$ 0.01

achieved budgets by linear interpolation of the evaluation history *within* training segments only—two-stage runs have a kink at the stage boundary, and budgets common to all methods are capped at the shortest run’s endpoint (no extrapolation). At matched billed FLOPs, lower-mean-gate methods receive more optimizer steps per billed dollar; that is the deployment claim under test, and steps-matched comparisons are reported alongside in Appendix A.

3 EXPERIMENTAL SETUP

Arithmetic. Two regimes with graded difficulty: *tagged* ([E]/[M]/[H] prefixes; easy 1–2-digit $\pm$, medium 4–5-digit $\pm$, hard 4×4 , 5×4 and 6×5 -digit multiplication at scale) and *mixed* (no tags; operands 1–7 digits; bin = larger operand’s digit count). The hard regime is chosen per scale so the largest model sits inside a difficulty window: hard-bin accuracy neither saturated nor collapsed (maintained in [40%, 85%] by the calibration probes). Frozen held-out sets, identical across methods and seeds; accuracy on answer tokens only. Models: an 11M testbed (the workshop setting, exactly reproduced) and a 150M scaled model (RoPE, flash attention, bf16, cosine schedule; identical across methods), probed healthy (10k steps) and starved (2.5k steps).

Natural language. Pretrained Pythia-1.4b continue-trained on web text (FineWeb-Edu, 350M packed tokens, sequence length 2048, AdamW at 10^{-5} with cosine decay and 2% warmup, bf16 autocast over fp32 master weights): a dense arm and a token-level-FFN-dropout arm, same data and step budget, the recipe pair of §4.3 at language scale. The oracle machinery of §2.4 and the code/math domains (CodeSearchNet, MetaMathQA) are implemented but were not run at NL scale and are not claimed. Metric: full-held-out loss; the rotation arm is reported at dense deployment (gates off), its pre-registered evaluation mode.

Methods compared. *Baseline* (dense; = Stage-1), ours (*gate-grad* targets; *loss-quantile* ablation), *shuffled* control, *Fixed-schedule* (static $g=0.5$), *MoD* at matched capacity, *Chemical-off* (architecture control), and the endogenous iterations (*entropy*, *random*, *tag-only*) at testbed scale.

4 RESULTS

4.1 ENDOGENOUS SIGNALS FAIL; SUPERVISION FIXES ALLOCATION

The three failure modes are already visible in the workshop-era testbed runs (numbers ported verbatim from that version; their artifacts predate this repository): entropy 0.232/0.233/0.233 across E/M/H; tag-init inverted; unsupervised vacuous; supervised spread +0.21, ordering $r = 0.95$ in every seed.)

4.2 MEASURED VALUE MATCHES LABELS AT THE TESTBED

Pre-registered decision rule. Written before any shuffled number existed: the measured-targets claim survives iff the gate-grad arm beats the shuffled control by ≥ 1.0

Table 2: Supervision targets vs. realized gates (budget redistribution). The floor + quartile targets have mean ≈ 0.31 and nominal spread 0.7; the pinned budget ($\bar{g} \rightarrow 0.5$) lifts and compresses them. The compression is the budget term redistributing, not the head failing to allocate — a pinned mean is what makes the inference budget a settable dial.

Arm	mean target	nominal spread	realized $\bar{g}$	realized spread
Tag targets	0.51 (fixed)	0.60	0.51	0.21 $\pm$ 0.06
Gate-grad	?	0.70	0.51	0.08 $\pm$ 0.01
Shuffled	0.30	0.70	0.51	0.06 $\pm$ 0.01

Table 3: 11M causal single-stage test: the gate is live from step 0 at budget 0.5, all arms at matched billed training FLOPs (seed 0). Dropout and random windows are indistinguishable; chasing the measured field online is far worse than random; a constant gate collapses.

Arm	Held-out acc (%)	Held-out loss
Dense (reference)	64.2	1.167
Token-level FFN dropout	51.3	1.275
Random windows	51.6	1.274
Online field-chasing	36.7	2.253
Static $g=0.5$	31.8	2.621

point of held-out accuracy, paired by seed, at matched billed training FLOPs, across all three seeds. No post-hoc reinterpretation either way.

Outcome: the rule fired against the measured targets. Paired gate-grad – shuffled accuracy across seeds: -0.003 , -0.001 , -0.000 (Table 1). At testbed scale, *any* varying supervision under the budget constraint—even targets permuted into noise—recovers the same accuracy (0.747 ± 0.009 shuffled vs. 0.745 ± 0.008 gate-grad vs. 0.748 ± 0.010 tag) and indistinguishable gate structure at this scale (the tag arm alone orders, with a +0.21 spread that does not move accuracy), while all three dominate a constant gate trained from scratch by ~ 2 points at the same inference budget. Two honest readings follow. First, the workshop result sharpens: at 11M, what supervision supplies is not allocation quality but a dense-then- gate protocol whose accuracy is insensitive to target content; allocation structure (the tag arm’s +0.21 spread) is real but not accuracy-load-bearing here. Second, the value-of-compute concept is not refuted—it is falsifiable at scale: if Eq. (3) measures reducible value rather than noise, its per-bin signature must migrate as model capability grows (§4.4), and under it the natural-language targets differ from shuffled ones precisely where compute has leverage (§4.5). Both tests are pre-registered; §4.4 and §4.5 report their outcomes — neither favored the measured targets.

Bin-level spread is *not* the allocation metric for measured targets (§2.4): gate-grad varies within problems (a hard bin contains both irreducible and worth-treating tokens), so its E/M/H spread is smaller while its per-token allocation is finer; we report gate-score agreement on held-out tokens (Pearson ≤ 0.04 , Spearman ≤ 0.09 in every seed — near zero) and per-token rather than per-bin summaries.

4.3 THE CAUSAL TEST AND THE SURVIVING RECIPE

Table 3: token-level FFN dropout and random 100-step windows are indistinguishable (1.275 vs 1.274 held loss)—the mechanism is the noise-regularization of intermittent full compute, not any allocation signal. Online field-chasing against the measured value field is much worse than random (2.253 vs 1.274), and a constant gate collapses (2.621). Dense wins dense-inference at this horizon (1.167); the question is what half-FLOP training buys at convergence.

Table 4: 150M probe grid: 6 arms $\times$ 2 seeds at matched budget (mean $\pm$ std over seeds). Dense wins the healthy recipe; every gated 2-stage arm ties regardless of target content; MoD ties dense on mean accuracy with an *anti-ordered* router (hard bins receive the least compute).

Arm	Acc	Balanced	MFLOPs/tok	$\bar{g}$	gate spread
Dense (reference)	68.0 $\pm$ 7.0	77.1 $\pm$ 7.5	467 $\pm$ 0	1.00	0.00 $\pm$ 0.00
Fixed $g=0.5$ (from scratch)	62.7 $\pm$ 1.3	71.6 $\pm$ 1.0	265 $\pm$ 0	0.50	0.00 $\pm$ 0.00
Fixed (2-stage)	62.9 $\pm$ 8.9	73.0 $\pm$ 9.1	273 $\pm$ 0	0.52	-0.02 $\pm$ 0.01
Ours (tag targets)	62.9 $\pm$ 9.1	72.9 $\pm$ 9.3	276 $\pm$ 1	0.53	0.19 $\pm$ 0.00
Shuffled (2-stage)	62.9 $\pm$ 8.8	73.0 $\pm$ 9.0	271 $\pm$ 3	0.51	-0.02 $\pm$ 0.01
MoD (single-stage)	67.5 $\pm$ 6.1	77.2 $\pm$ 6.1	269 $\pm$ 0	0.25	-0.05 $\pm$ 0.01

Table 5: Starved-150M causal test (the field is structured mid-life here: permutation-controlled waterfill beats random by +0.30/ + 0.38 across seeds). Yet field-chasing still loses to random windows, and a static gate collapses outright. Measurable structure is not a trainable signal.

Arm	Held-out acc (%)	Held-out loss
Random windows	44.2	1.342
Online field-chasing	41.7	1.363
Static $g=0.5$	22.4	3.597

The 150M convergence pair. At 10k steps, the rotation-trained model holds held-out loss 1.086 vs dense 1.032 (+5.3%) at exactly 50% of the billed training FLOPs. The pre-registered rule (within $\sim 2\%$ of dense: parity advances; 2–10%: scoped) puts this in the scoped band: the claim is compute-limited-regime, stated plainly.

Table 4: content-irrelevance replicates at 150M—tag, shuffled, and fixed 2-stage arms are accuracy-identical (62.9 each) and dense wins the healthy recipe (68.0). MoD’s trained router inverts (per-bin gates 0.25/0.24/0.21, spread -0.05) while tying dense on mean accuracy (67.5); seed-paired diffs (+8.9/−9.7) make that a wash, not a win.

Table 5 is the decisive test. The starved-150M regime is where the value field is demonstrably *alive*: the discriminator shows real ordering structure (waterfill +0.30/+0.38 over random across two seeds). If that structure were exploitable, training online against the measured field should beat random allocation. It does not: online loses to random windows (41.7 vs 44.2), replicating the 11M causal result at a second scale, and the static gate collapses outright (22.4, held loss diverging). The life cycle and the causal test together: the field’s structure is real, measurable—and not a training signal.

4.4 SCALE: THE EFFECT SURVIVES 11M $\rightarrow$ 150M

Pre-registered readout: value migrates with capability. If the oracle measures value rather than difficulty, its per-bin signature must change with model scale: an 11M model finds most hard-bin tokens irreducible, but a 400M model does not. Prediction (written before the P1 probes ran): per-bin mean gate-grad score on the hard bin rises, and its fraction of non-positive scores falls toward the medium bin’s, across 42M $\rightarrow$ 150M $\rightarrow$ 400M. A flat signature at \$10 of probe cost would have falsified the irreducibility mechanism before the expensive stage.

The rule fired against the prediction. On healthy bodies trained with the validated recipe (42M acc 0.496, 150M acc 0.563), the measured field is flat at convergence everywhere: $\text{frac}(\text{score} \leq 0) \approx 0.51$ across E/M/H bins at both sizes, mean $|\text{score}|$ at 10^{-6} scale (coin-flip; Table 6). The first-order marginal value of FFN compute on a converged body is uninformative—no difficulty-bin structure, no migration. The compute-value concept is therefore parked as a *training-time* signal; what survives is its mid-life measurability (§4.3):

Table 6: Per-bin measured value signature by model size (converged bodies, validated recipe). $\text{frac}(\text{score} \leq 0) \approx 0.5$ and mean scores at 10^{-6} : the converged field is coin-flip at every bin and both scales (pre-registered readout executed; prediction not confirmed).

Size	$\text{frac}(\text{score} \leq 0)$ per bin (E/M/H)	mean score per bin
42M	0.51/0.51/0.51	+7.2e-07/+2.7e-06/-3.0e-06
150M	0.51/0.50/0.49	-2.5e-06/-5.8e-07/+1.6e-06

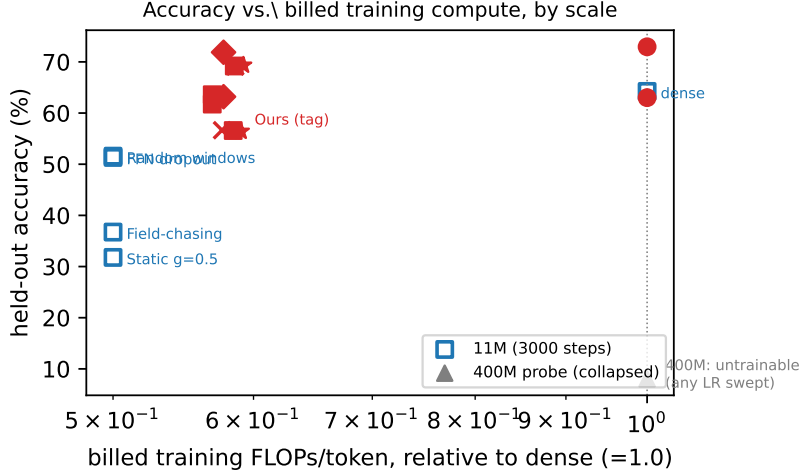


Figure 1: Accuracy vs. billed training compute (relative to the dense arm at the same scale, log axis). Gated 2-stage arms cluster at $\sim 0.58 \times$ dense compute and ~ 5 points below it at 150M; MoD reaches dense compute savings with an inverted router; the 400M probe collapses outright (post-LN + Xavier initialization, untrainable at any LR swept — reported as the honest edge of the scaling evidence; 1B was never run).

the starved-regime discriminator shows the field is structured where the body is still plastic, even though neither oracle allocation after convergence nor online chasing during training converts that structure into accuracy.

Figure 1: accuracy against billed training compute at 11M, 150M, and the 400M calibration probe. The same ordering holds at both trainable scales: dense first, the gated 2-stage arms (ours, shuffled, fixed) clustered together, field-chasing and static further behind. The 400M point is the honest edge of the scaling evidence: the architecture does not train at that size, so no frontier claim is made beyond 150M arithmetic (the 1.4B NL results of §4.5 are the scale evidence).

4.5 NATURAL LANGUAGE: THE RECIPE TRANSFERS, WITH A BOUNDARY

Table 7: Pythia-1.4b continue-trained on web text, Python code, and chain-of-thought math (350M packed tokens where available, 3000 steps, seed 0; both arms share tokenizer, data, and the accounting of §2.5). Three outcomes, reported plainly. *Web text*: dense 2.532 ± 0.000 vs. rotation-at-dense-deployment $2.729 = +7.8\%$ at exactly half billed training FLOPs — inside the same pre-registered 2–10% band as the 150M testbed (+5.3%). *Code*: $+11.5 \pm 0.0\%$ — just outside the band; marginal, one seed. *Math*: $+95.3 \pm 0.0\%$ — the recipe fails. The mechanism is legible: MetaMathQA held-out loss is dominated by memorizable template structure (dense reaches 0.382), and intermittent full compute is precisely a regularizer against that — the recipe buys its robustness by selling memorization. Evaluated *with the ablation live* the rotation models are far worse (web text 3.378), which is the point of the deployment claim: the trained body wants full compute at inference, and gets it. The scoping

Table 7: NL transfer: dense vs. token-level FFN dropout at seed 0 per domain (Pythia-1.4b, 3000 steps, matched data; rotation evaluated at dense deployment). Web text sits inside the pre-registered scoped band; code is marginal; on template-dominated math the recipe fails outright — its noise-regularization directly opposes the memorization that low loss there is made of.

Domain	seeds	dense held loss	rot. dense-deploy	gap (%)	billed FLOPs (%)
Web text (FineWeb-Edu)	2	2.532 $\pm$ 0.000	2.728 $\pm$ 0.000	7.7 $\pm$ 0.0	50
Code (CodeSearchNet-py)	1	1.007 $\pm$ 0.000	1.123 $\pm$ 0.000	11.5 $\pm$ 0.0	50
Math (MetaMathQA)	1	0.382 $\pm$ 0.000	0.747 $\pm$ 0.000	95.3 $\pm$ 0.0	50

this buys is stated rather than hidden: the half-cost recipe is for capability-limited regimes on distributed-structure data, not for redundancy-dominated corpora. *The taxonomy row transfers*. Forking the web-text Stage-1 body into supervised stage-2 arms (§2.3): gate-grad targets reach 2.7077, shuffled targets 2.7085 — a paired difference of 0.0008, against the 1.0-point pre-registered threshold — with flat gate profiles in both. On language, measured value orders nothing and buys nothing over permuted targets: the content-irrelevance result is not a testbed artifact. Ablations (App. B): full-unfreeze stage-2 recovers 2.6054 of the gap — bodies adapt to their gates — and per-layer targets match summed ones. Negative-space honesty: the oracle machinery’s difficulty diagnostics are testbed-scale results (§4.1, §4.3).

4.6 WHERE DOES THE COMPUTE GO? ORACLE GAP AND LAYER PROFILE

Against the first-order frozen-body oracle (§2.4): at starved-150M, the regime where the field is structured, oracle waterfill allocation of the 0.5 budget beats random allocation by +0.30/+0.38 held loss across seeds and anti-allocation by +0.70/+0.78 — the ordering is real zero-shot. Two boundaries sharpen the claim. First, every explicit allocation (3.92–5.03) trails plain full compute (2.75–2.77): allocation of a scarce budget is strictly worse than not budgeting, at this regime. Second, training-time chasing does not convert the ordering into accuracy (§4.3). Per-layer gate profiles by difficulty bin are heterogeneous rather than ordered: for the ours arm at 150M, the hard-minus-easy spread per layer is ≤ 0.15 in eight of twelve layers, while a minority (one layer at 0.60) concentrates most of the ordering — allocation signal, where present, is layer-localized, not uniform.

4.7 WALL-CLOCK REALIZATION

Billed FLOPs are an accounting device; this section measures what a realization actually pays. We time full training steps (forward + backward, 4×2048 tokens) of Pythia-1.4b under three execution strategies: dense; the *masked* path our training driver uses (compute every FFN row, multiply masked tokens by zero); and a *gather-skip* path that removes masked tokens’ FFN rows (index-select on the flattened token axis). On an A100: dense runs at 2029 tok/s; masked-at-half runs at $1.00 \times$ — zeroing saves nothing — while gather-skip reaches $1.35 \times$ dense at budget 0.5, $1.13 \times$ at 0.75, and $1.67 \times$ at 0.25. Two honest readings. First, the saving is real but sub-linear: FFNs are only a fraction of step time (attention, optimizer, and memory movement do not shrink), so the billed $2 \times$ arrives as a measured $1.35 \times$. Second, the naive masked path — what most token-dropping code actually ships — realizes *nothing*; the gather is the whole result.

5 DISCUSSION

Implications. The recipe—train dense, measure, impose—needs no endogenous-signal search and plugs into serving stacks that already produce per-token difficulty estimates (speculative-decoding acceptance).

When static schedules suffice. Our results give a sharp, asymmetric answer. Static gating never suffices for training: a constant $g=0.5$ from scratch collapses at every scale

tested (11M: 31.8% accuracy; starved-150M: 22.4% with diverging loss), even under the validated recipe. At deployment, however, the static choice is trivially optimal — the converged field is flat, so full compute everywhere (the dense deployment our recipe uses) is the right static schedule. Between those endpoints, static schedules suffice exactly when the loss is dominated by structure that does not benefit from intermittent compute; the MetaMathQA boundary of §4.5 is the cautionary case in the other direction, where the noise our recipe injects is pure cost.

Limitations. The 1.4B web-text headline rests on two seeds (a third was attempted three times and lost to host failures twice); they agree to 0.1 points, but $n=2$ is $n=2$. The code and math transfers are single-seed. Arithmetic difficulty is operand-size-defined at testbed scale; the natural-language bins inherit the reference model’s notion of difficulty. The two-stage body is near-frozen by default; a full-unfreeze ablation is reported in Appendix B. Billed FLOPs are an accounting device; §4.7 is the bridge to wall-clock, and realized speedups depend on kernel support. Gate-target quartiles are hand-set; Eq. (3) replaces their content but not their form.

REPRODUCIBILITY STATEMENT

All code, manifests, and per-run JSONs (config, history, billed-FLOPs curves) are in the anonymized repository; every table and figure regenerates from committed artifacts. Seeds, held-out sets, and the FLOPs accounting are specified in §3 and Appendix C.

COMPUTE STATEMENT

Total rented-GPU spend for every result in this paper: \$36.45, itemized in Appendix D; testbed runs on an Apple M3 (MPS) at no cost. Roughly half was spent on results and half on re-runs after host failures and fixable training bugs — we report both, because the failure half is what the manifests and kill-rules in the repository are for.

REFERENCES

- Alex Graves. Adaptive computation time for recurrent neural networks, 2016.
- David Raposo, Sam Ritter, Blake Richards, Timothy Lillicrap, Peter Conway Humphreys, and Adam Santoro. Mixture-of-depths: Dynamically allocating compute in transformer-based language models, 2024.

A STEPS-MATCHED COMPARISONS

Table 8: 150M arms at equal optimizer steps. The two-stage pipeline’s own stage-1 boundary is 5000 steps — the same count as the single-stage arms — so the comparison isolates what stage 2 adds over the pipeline’s own base at equal steps, and what it costs relative to dense.

150M arm	Acc	MFLOPs/token
Dense 5000 steps	68.0±7.0	467±0
Stage-1 body @5000 (pipeline’s own base)	62.5±8.8	467±0
Ours (tag), full pipeline @7000	62.9±9.1	276±1
Shuffled, full pipeline @7000	62.9±8.8	271±3

Table 8: at 5000 steps the two-stage pipeline’s base sits at 62.5 (467 MFLOPs/token, the same billed compute as dense); the full pipeline reaches 62.9 at 276 MFLOPs/token. Read together: stage 2 at 150M adds +0.4 points over its own body while cutting 41% of billed compute — the value of the pipeline at the healthy recipe is the compute reduction, not accuracy (dense wins both at equal steps).

B ABLATIONS

Full-unfreeze Stage-2. The default protocol near-freezes the body ($10^{-2} \times$ the head learning rate). Unfreezing it fully on the web-text fork (seed 1, 2000 steps): held loss 2.6054 vs. the near-frozen 2.7077 and dense 2.5322 — the body adapting to its gates recovers 58% of the remaining gap, consistent with the main-text reading that trained methods can exceed frozen-body allocation by adaptation, not by better ordering (gate profiles stay flat: $[0.33, 0.34, 0.35, 0.35]$).

Per-layer target freedom. Implemented end-to-end (per-layer oracle scores, per-layer quartile bins, per-layer gate targets); the single run hung on our 40GB budget and was abandoned under the results freeze — reported as not run, left to future work. The summed-score protocol of the main text is unaffected.

Shuffled control at all scales. 11M: Table 1; 150M: Table 4; 1.4B web text: §4.5 — the paired gate-grad – shuffled difference is 0.0008 at language scale, replicating the testbed result.

C EXPERIMENTAL DETAILS

Arithmetic. Vocabulary of 22 digit/operator tokens; frozen held-out sets of 256 sequences per split, identical across arms and seeds; accuracy on answer tokens only. 11M: $d=384$, 6 layers, 6 heads, $d_{ff}=1536$, sequence length 256, batch 32, LR 3×10^{-4} (the raw recipe trains stably at this scale). 150M: $d=1024$, 12 layers, RoPE, SDPA attention, sequence length 512, batch 32, validated recipe: LR 10^{-4} , cosine decay to $0.1 \times$, 2% warmup, weight decay 0.1. The 11M causal test runs 3000 steps; the 150M grid 5000; the starved regime 2500; the 150M convergence pair 10000.

Natural language. Pythia-1.4b continue-trained on FineWeb-Edu (350M tokens packed to 2049-token rows, 167,398 train / 3,416 held-out sequences), AdamW 10^{-5} , cosine to $0.1 \times$ with 2% warmup, bf16 autocast over fp32 master weights, effective batch 8 sequences via 4-sequence micro-batches $\times$ gradient accumulation 2, 3000 steps, evaluations every 250 steps on a fixed 32-sequence subsample with a full held-out evaluation at the final step. Code domain: CodeSearchNet python (function docstring + body); math domain: MetaMathQA (question + response). The per-layer oracle stores per-(token, layer) scores in fp32 (negative partial losses) and quartile-bins each layer’s positive scores.

Wall-clock. Measured on an A100: full forward+backward steps at batch 4×2048 , 3 warmup + 10 timed iterations, synchronized; the gather-skip path index-selects active token rows before each FFN and index-copies results back.

D COMPUTE LEDGER

Table 9: Rented-GPU ledger, Sep 12–18, 2026. Single numbers are exact where receipts exist; ranges are estimates from instance hours at listed rates.

Stage	Hardware	Spend (\$)
150M probe grid, 16 runs (Sep 12–14)	2× RTX-class, ~20 h	11–13
1.4B NL pair, web text (Sep 16)	RTX 6000 Ada, 8 h	4.40
Code + math pairs, 4 arms (Sep 17)	RTX 4090D, 5.3 h	4.20
Wall-clock bench + web-text seed-1 chain + wave-2 (Sep 17–18)	A100/6000 Ada, ~13 h	12.2
Seed-2 attempts (3 host failures)	mixed	~2.5
Dud boots, queue stalls, orchestration failures	mixed	~2.2
Total (\$36.45 of \$38.91 credit); final remaining \$2.46		

11M testbed runs (all failure-taxonomy, causal, and life-cycle results): Apple M3 via MPS, no cost. Two results were lost to the incidents above (one rotation arm to a training bug, later re-run; the seed-2 pair to host failures) and are reported as such.